

ZADANIE: Strona logowania (Angular 18.2 + TypeScript)

Porównywanie loginu i hasła z pliku `tajne_dane.json`

Czas: 1h lekcyjna

Cel zadania

Zbudowanie prostej strony logowania.

Cele dodatkowe:

- uczeń potrafi pobrać dane z pliku za pomocą `HttpClient`
- uczeń potrafi wczytywać dane z JSON do obiektu
- uczeń ćwiczy umiejętność używania tablic

Pliki i materiały

- 1) Plik z danymi: `16.5_tajne_dane.json` (dołączony do zadania)
- 2) Projekt Angular 18.2 (standalone)
- 3) Moduły/feature: `HttpClient`, formularz (Reactive Forms lub template-driven)

Format pliku `tajne_dane.json`

W pliku znajduje się JSON o strukturze:

```
{
  "users": [
    { "username": "anna", "password": "Haslo123!" },
    { "username": "adam", "password": "Qwerty!23" }
  ]
}
```

Kroki do wykonania

1. Umieść plik `tajne_dane.json` w folderze: `src/assets/` (aby Angular mógł go wczytać).
2. Dodaj w aplikacji formularz logowania: pola login i hasło oraz przycisk „Zaloguj”.
3. Po kliknięciu „Zaloguj” wczytaj plik `tajne_dane.json` przez `HttpClient`.

4. Zamień tekst na obiekt (JSON.parse) i pobierz tablicę users.
5. Sprawdź, czy istnieje użytkownik o podanym username oraz czy password jest zgodne.
6. Wyświetl komunikat: „Zalogowano” albo „Błędny login lub hasło”.
7. Dodatkowo: zablokuj przycisk, jeśli pola są puste; pokaż krótkie walidacje.

Uwaga (bezpieczeństwo)

To ćwiczenie jest tylko edukacyjne. W prawdziwych aplikacjach NIE przechowuje się haseł w plikach w aplikacji front-end. Logowanie powinno odbywać się przez serwer (API), a hasła powinny być hashowane.

Włącz HttpClient w aplikacji - nowe

Edytuj: (Uwaga: nic nie kasuj w tym pliku – tylko dopisz czego brakuje)

src/main.ts

```
import { bootstrapApplication } from '@angular/platform-browser';
import { provideHttpClient } from '@angular/common/http';
import { AppComponent } from './app/app.component';
bootstrapApplication(AppComponent, {
  providers: [provideHttpClient()],
}).catch(err => console.error(err));
```

Żeby Angular mógł robić zapytania HTTP, musisz “włączyć” klienta HTTP.

W main.ts

```
providers: [provideHttpClient()]
```

Co to robi?

- rejestruje w aplikacji usługę HttpClient
- dzięki temu możesz ją wstrzyknąć w konstruktorze komponentu:
constructor(private http: HttpClient) {}

Użycie w kodzie głównym aplikacji:

```
constructor(private http: HttpClient) {}
```

Typ danych (interfejs)

Plik JSON ma strukturę np.:

```
{ "users": [ { "username": "...", "password": "..." } ] }
```

Tworzysz interfejs w nowym pliku, żeby TS podpowiadał pola i pilnował struktury: (tablica)

```
export interface SecretFile {  
  users: { username: string; password: string }[];  
}
```

Po co?

- autouzupełnianie (data.users, u.username)
- mniej błędów literówek
- czytelniejszy kod

To nie zmienia działania programu, tylko pomaga w pisaniu

Pobranie danych: http.get(...)

Najważniejsza linia:

```
this.http.get<SecretFile>('assets/tajne_dane.json')
```

Co to znaczy?

- `get<SecretFile>(...)`
mówi: *“pobieram JSON i chcę go traktować jak SecretFile”*
- `'assets/tajne_dane.json'`
to adres pliku w aplikacji (po uruchomieniu działa jak URL)

Co zwraca get(...)?

Zwraca **Observable** (strumień danych), czyli wynik **asynchroniczny** (przyjdzie “za chwilę”).

Dlatego musisz “odebrać” wynik przez `subscribe(...)`.

5) Odbiór wyniku: subscribe

Przykład:

```
this.http.get<SecretFile>('assets/tajne_dane.json').subscribe({  
  next: (data) => {  
    // tu masz dane z pliku JSON
```

```
    },  
    error: () => {  
      // tu wchodzisz, gdy np. plik nie istnieje (404)  
    }  
  });
```

Co oznacza next?

- wykona się, gdy pobranie się uda
- data to już gotowy obiekt (Angular sam parsuje JSON)

Co oznacza error?

- plik nie znaleziony, zła ścieżka, brak dostępu itd.

Co dalej robisz z data?

Masz w `data.users` tablicę użytkowników. Możesz wyszukać użytkownika:

- ```
const user = data.users.find(u => u.username === this.login);
```
- `find(...)` przeszukuje tablicę i zwraca pierwszy pasujący obiekt albo `undefined`

Potem porównujesz hasło:

```
if (user && user.password === this.password) {
 // OK
} else {
 // BŁĄD
}
```

UWAGA: wygląd robimy za pomocą bootstrapa!!!